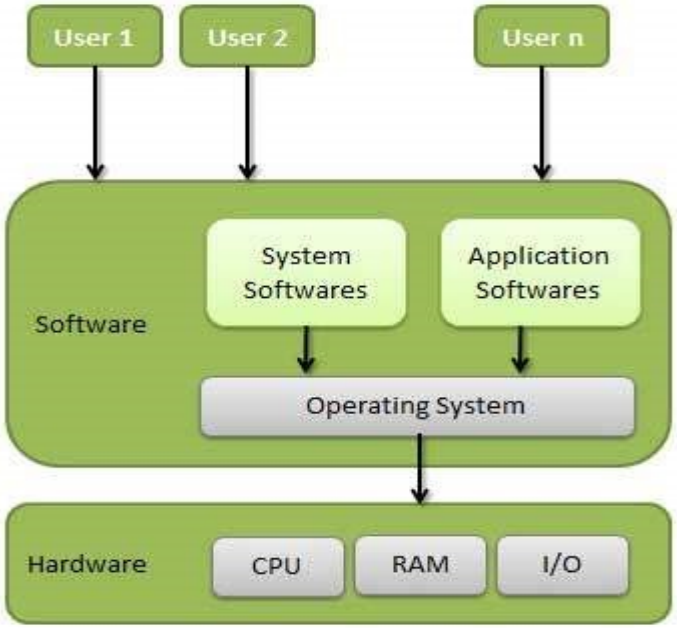
Introduction to Operating System

- An **Operating System (OS)** is system software that acts as an **interface between the user and the computer hardware**. It manages all the resources of a computer system and provides an environment in which user programs can execute efficiently and conveniently.
- An **Operating System** is a program that:
- Controls the execution of application programs
- Manages computer hardware resources such as CPU, memory, storage, and input/output devices
- Provides services to users and applications
- **Example:** Windows, Linux, UNIX, macOS, Android

Operating System as an Interface
The OS acts as a **bridge**:
User → Operating System → Hardware
Users interact with applications, applications request services from the OS, and the OS communicates with hardware.



TYPES OF OPERATING SYSTEMS

- Batch Operating System
- Iterative Operating System (IOS)
- Time Sharing OS
- Multiprocessor OS
- Distributed OS
- Cluster Operating System
- Real Time OS

Batch Operating System

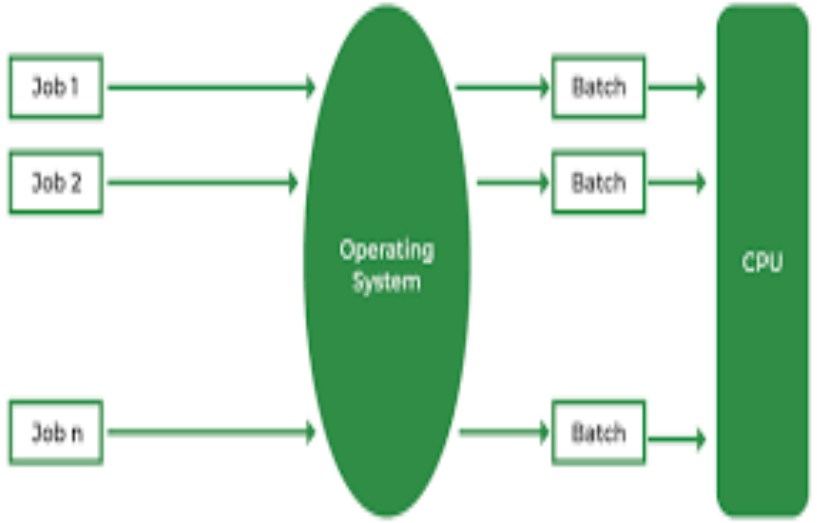
Definition

- A Batch Operating System processes jobs in batches. The user submits jobs, and the OS executes them sequentially, without any manual interaction during execution.

Example

Bank Payroll System

- Calculates salaries for thousands of employees
- All salary calculation jobs are collected.
- Executed together at night.
- Output is salary slips.



Advantages

✔ Efficient for large repetitive tasks
✔ Easy to manage

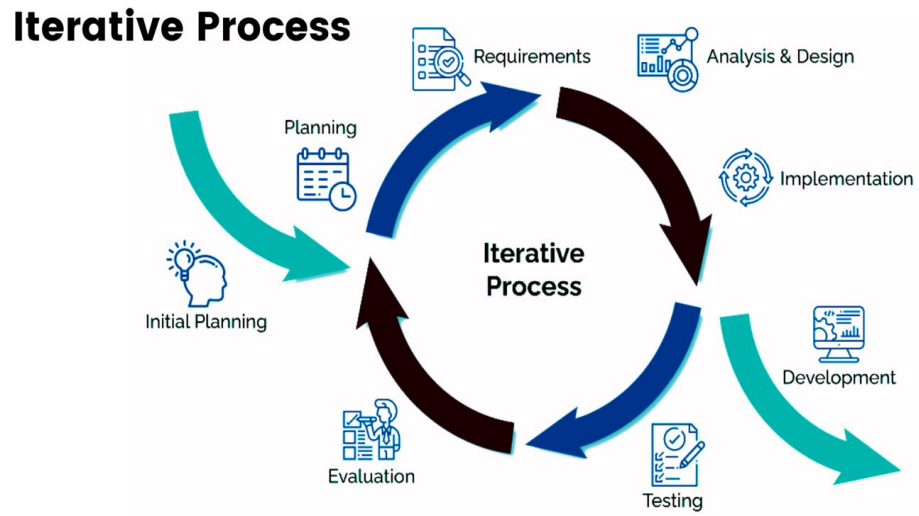
Disadvantages

✖ Long turnaround time
✖ Difficult to debug errors

Iterative/Multiprogramming Operating System (IOS)

Definition

- An Iterative Operating System allows direct interaction between the user and the computer.
  The system accepts user input, processes it immediately, and gives quick responses.
- **Examples**
- Windows
- Android
- ATM systems
- Railway reservation systems



- **Features**

1. Time Sharing: CPU time is divided among multiple users/programs.
2. Fast Response: System responds within seconds or milliseconds.
3. User-Friendly Interface Supports: GUI (icons, menus) CLI (commands)
4. Multitasking: Multiple applications run at the same time.

Advantages

✔ Immediate response
✔ Easy to use
✔ Suitable for real-time user work
✔ Better productivity

Disadvantages

- ✖ Requires more memory
  ✖ Complex design
  ✖ Performance may reduce with many users

Time Sharing/Multitasking OS

Definition

A Time Sharing Operating System allows **multiple users or multiple programs** to share the CPU simultaneously.
Each process is given a small slice of CPU time (time quantum), and the CPU switches rapidly between processes.

Real-life example:

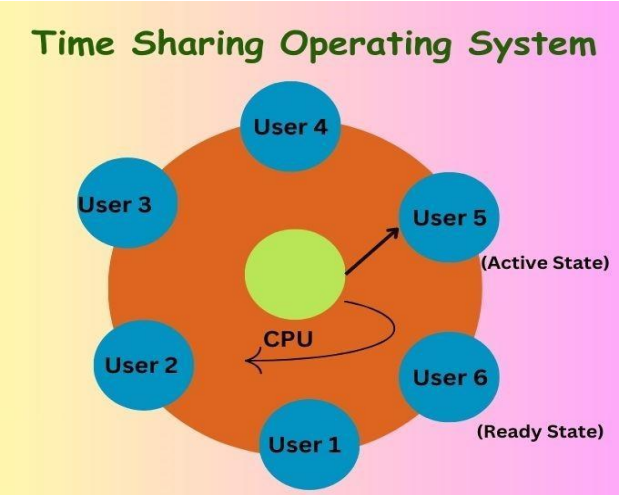
A college computer lab:

Many students use one server

Everyone types commands

Each student gets CPU time in turns

Output appears quickly



Advantages

✔ Quick response to users
✔ Efficient CPU utilization
✔ Fair sharing of CPU
✔ Supports multitasking

Disadvantages

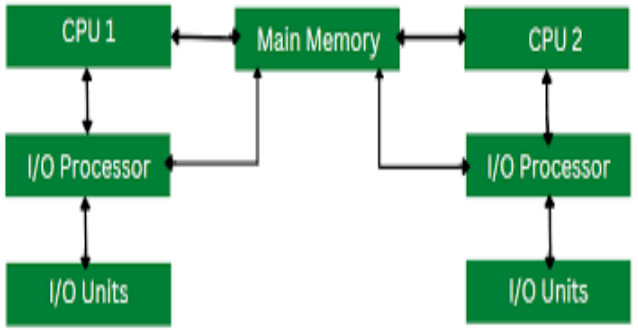
✖ Performance decreases if many users
✖ Requires high memory

Multiprocessor OS

- **Definition**

A Multiprocessor Operating System is an OS that uses two or more CPUs/processors within a single computer system to execute processes simultaneously.

- All processors share common memory, I/O devices, and OS resources.
- Types of Multiprocessor Systems
- 1. Symmetric Multiprocessing (SMP) :All processors are equal Same OS runs on all CPUs, Any processor can execute any process
- Example:
  Modern multicore PCs (Intel i5, i7)



2. Asymmetric Multiprocessing (AMP)
One master processor controls others, Master assigns tasks to slave processors
Slaves perform specific jobs, Example : Embedded systems, older mainframes

- Advantages

✔ Faster processing
✔ High system throughput
✔ Better resource utilization
✔ System continues even if one CPU fails

- Disadvantages

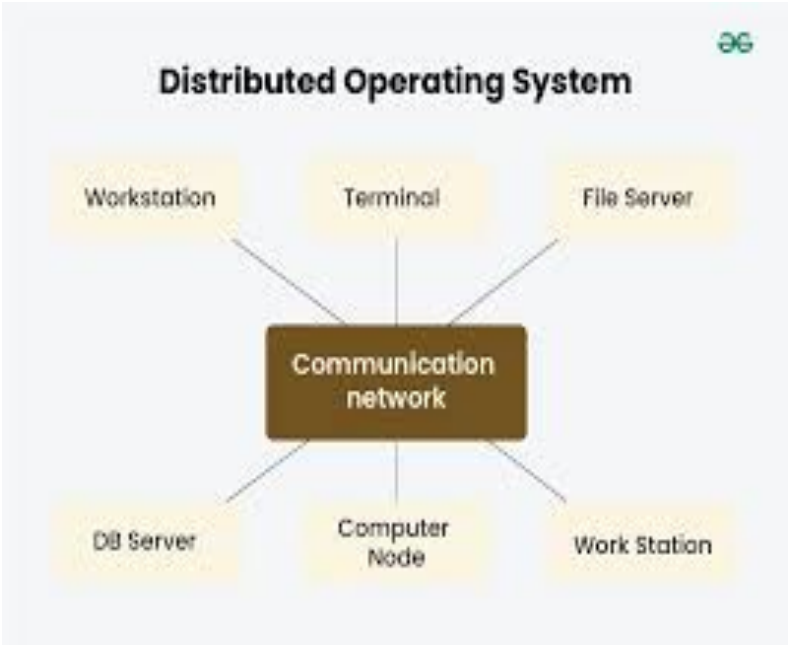
✖ Complex OS design
✖ Expensive hardware
✖ Requires advanced scheduling

Distributed/Network Operating System

Definition

A Distributed Operating System is an operating system that manages a group of independent computers and makes them appear to users as a single unified system.

Each computer has its own processor and memory, but they communicate through a network.



Types of Distributed Systems 1. Client–Server Model

Server provides services : Clients request services

Example: Web servers

2. Peer-to-Peer Model

All nodes act as equals :No central control

Example: File sharing networks

Advantages

✔ High reliability
✔ Resource sharing
✔ Scalability
✔ Improved performance

Disadvantages

✖ Network dependency
✖ Complex design

Cluster Operating System

Definition

A Cluster Operating System manages a group of independent computers (nodes) that work together as one system to provide high performance, high availability, or load balancing.

Each node has its own OS, but the cluster software coordinates them.

Types of Cluster Systems

1.**High Availability (HA) Cluster:** Focuses on fault tolerance

- Backup node takes over if primary fails (Example: Banking servers)

2. **Load Balancing Cluster** : Workload is shared across nodes

Improves response time Example: Web servers

(Google, Amazon)

Clustered System

3. **High Performance Computing (HPC) Cluster**
Used for heavy computations Tasks divided and processed in parallel
Example: Weather forecasting systems

- **Advantages**

✔ Very high performance
✔ Reliable and fault tolerant
✔ Scalable (easy to add nodes)
✔ Better load management

- **Disadvantages**

✖ Expensive setup
✖ Complex management
✖ Requires skilled administration
✖ Network dependency

Real-Time Operating System (RTOS)

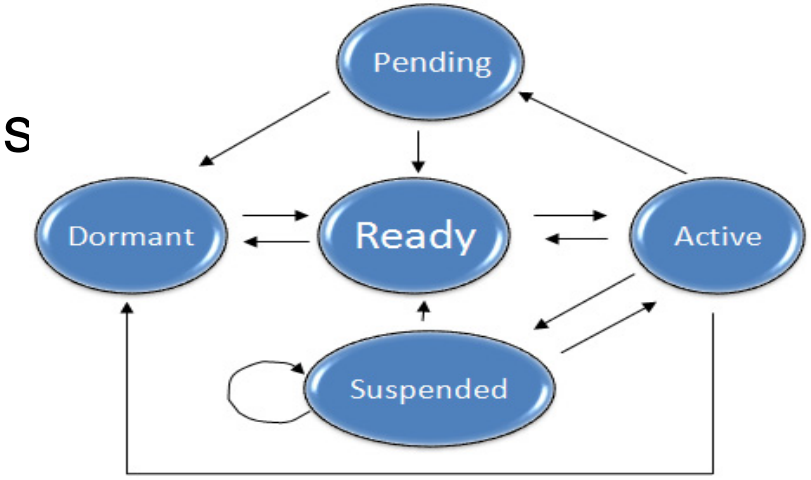
Definition

A Real-Time Operating System is an operating system that guarantees a response within a fixed time limit.
In RTOS, correctness depends on both the result and the time at which the result is produced.

Missing a deadline = system failure (for many applications).

Types of Real-Time Operating Systems

1. Hard Real-Time OS
2. Soft Real-Time OS



1. Hard Real-Time OS

- Strict deadlines
- Missing a deadline is unacceptable

Examples:

- Aircraft control systems

2. Soft Real-Time OS

Deadlines are important but occasional misses are allowed

Examples:

- Video streaming
- Online gaming
- Multimedia systems

Advantages

✔ Guaranteed response time
✔ High reliability
✔ Efficient handling of time-critical tasks

Disadvantages

- ✖ Complex design
  ✖ Limited user interface
  ✖ Expensive development
  ✖ Less flexible than general OS

UNIX Operating System – Introduction

Definition

UNIX is a multi-user, multitasking, time-sharing operating system originally developed at AT&T Bell Labs.

It is designed to be portable, secure, and stable.

History (Short)

Developed in 1969

Creators: Ken Thompson & Dennis Ritchie

Written mainly in C language

Became the foundation for Linux, macOS

- **Architecture of UNIX OS**

Layered Structure

- **User**

↓

- **Shell**

↓

- **Kernel**

↓

- **Hardware**

1. Kernel Core of UNIX

Controls hardware

Manages:CPU,Memory,Files,Devices,Processes

2. Shell Interface between user and kernel

- Interprets user commands
- Types of Shells:
- Bourne Shell (sh)
- Bash
- C Shell (csh)
- Korn Shell (ksh)

3. File System

- Organizes data in hierarchical tree structure
- Everything is treated as a file
- Example:
- /
- |-- bin
- |-- home
- |-- etc
- |-- usr

- **Features of UNIX**

🔷 Multi-User

- Multiple users can access the system at the same time.

🔷 Multitasking

- Multiple processes run simultaneously.

🔷 Portability

- Can run on different hardware platforms.

🔷 Security

- User authentication
- File permissions (read, write, execute)

🔷 Time Sharing

- CPU time is shared among users/processes.

UNIX File Permissions

- **r – read**
- **w – write**
- **x – execute**
- **Example:**
- **-rwxr-xr—**

Basic UNIX Commands

ls	List files
pwd	Show current directory
cd	Change directory
mkdir	Create directory
rmdir	Remove directory
cp	Copy files
mv	Move/rename files
rm	Delete files
cat	Display file content
who	Show logged-in users
date	Display date and time

Process Management in UNIX

Each process has a Process ID (PID)

Commands:

- ps – show processes
- kill – terminate process
- top – resource usage

Advantages of UNIX

✔ Very stable and reliable
✔ Strong security
✔ Supports networking
✔ Scalable
✔ Open-source variants available

Disadvantages of UNIX

✖ Mostly command-line based
✖ Not beginner-friendly

Applications of UNIX

- Servers
- Web hosting
- Databases
- Cloud systems
- Scientific research

PART–2: OPERATING SYSTEM STRUCTURES

1. Computer System Structure

A **computer system** is an integrated combination of **hardware and software** that works together to perform computations, store data, and execute programs.
The structure of a computer system explains **how different components are organized and how they interact** with each other

• 1. Basic Components of a Computer System

- A computer system mainly consists of:
- **Hardware**
- **Operating System**
- **Application Programs**
- **Users**

Diagram Representation (Conceptual)

- Users
- ↓
- Application Programs
- ↓
- Operating System
- ↓
- Hardware

- **2. Hardware Components**
- Hardware is the **physical part** of the computer system.
- **a) Central Processing Unit (CPU)**
- The CPU is the **brain of the computer**. It executes instructions and processes data.
- **Parts of CPU:**
- **Arithmetic Logic Unit (ALU)**
- Performs arithmetic operations (addition, subtraction)
- Performs logical operations (AND, OR, NOT, comparisons)
- **Control Unit (CU)**
- Controls the execution of instructions
- Coordinates activities of all hardware components
- **Registers**
- Small, high-speed storage inside CPU
- Store instructions, addresses, and intermediate results

- **b) Main Memory (Primary Memory)**
- Stores **programs and data currently in use**
- Directly accessible by the CPU
- Volatile in nature
- **Types:**
- **RAM (Random Access Memory)** – temporary storage
- **ROM (Read Only Memory)** – permanent storage

c) Secondary Storage

- Used for **long-term data storage**

Examples:

- Hard disk
- SSD
- Pen drive
- CD/DVD

- **d) Input Devices**
- Used to input data into the computer.
- Examples:
- Keyboard
- Mouse
- Scanner
- Microphone

- **e) Output Devices**
- Used to display results.
- Examples:
- Monitor
- Printer
- Speaker
- Projector

3. Operating System (OS)

- The **Operating System** acts as an **interface between the user and the hardware**.
- **Functions of OS:**
- Process management
- Memory management
- File system management
- Device management
- Security and protection
- Without an OS, hardware cannot be used effectively

4. Application Programs

- Application programs are **user-oriented programs** designed to perform specific tasks.
- Examples:
- Word processors
- Web browsers
- Media players
- Database software
- These programs **run on top of the operating system**

- **5. Users**
- Users are the people who interact with the computer system.
- Types of users:
- End users
- Programmers
- System administrators

- **6. Computer System Operation**
- **Instruction Execution Cycle**
- **Fetch** – Instruction is fetched from memory
- **Decode** – Instruction is decoded by control unit
- **Execute** – Operation is performed by ALU
- **Store** – Result is stored back in memory
- This cycle repeats continuously.

- **7. Storage Structure**
- Computer systems use a **storage hierarchy**:
- Registers (fastest, smallest storage locations inside CPU)
- Cache memory(located b/w CPU and MM)
- Main memory (RAM)
- Secondary storage (HDD/ SSD)

- **8. I/O Structure**
- Input/Output devices communicate with CPU using **device controllers**
- OS manages I/O operations
- Uses techniques like:
- Interrupts
- Direct Memory Access (DMA)

- **9. Multiprocessor Systems (Brief)**
- Use **more than one CPU**
- Share memory and devices
- Advantages:
- Reliability
- Scalability

- **10. Advantages of Computer System Structure**
- Efficient resource utilization
- Faster execution
- Better user interaction
- Supports multitasking and multiuser environments

Conclusion

The **computer system structure** defines how hardware, software, and users interact to perform tasks efficiently. Understanding this structure is essential for studying **Operating Systems**, system design, and computer architecture

Network Structure

- A **network structure** (also called **network architecture**) defines **how computers and devices are organized, connected, and communicate** with each other in a network.
  It specifies the **physical layout, logical design, communication methods, and protocols** used in data exchange.

1. Components of Network Structure

A computer network is made up of the following components:

- **1.1 Nodes**
- Devices connected to the network
- Can send, receive, or forward data
- Examples:
- Computers
- Servers
- Printers
- Routers

- **1.2 Transmission Media**

Medium through which data travels.

Types: Wired Media

  - Twisted pair cable
  - Fiber optic cable

- **Wireless Media**
  - Wi-Fi
  - Bluetooth

- **1.3 Network Devices**
- **Hub** – broadcasts data to all devices
- **Switch** – forwards data to specific device
- **Router** – connects different networks
- **Modem(modulator & demodulator)** – connects to ISP(Internet service provider)
- **Access Point** – enables wireless access

I/O Structure in Operating System

- **Introduction**

The **Input/Output (I/O) structure** in an Operating System defines **how input and output devices interact with the CPU, main memory, and the OS**.
Since I/O devices are slower than the CPU, the OS provides efficient mechanisms to **control, coordinate, and optimize I/O operations**.

1. I/O Devices

- I/O devices are used to **input data into the system or output results**.
- **Examples:**
- Input devices: Keyboard, mouse, scanner
- Output devices: Monitor, printer, speaker
- Storage devices: Hard disk, SSD, USB drive
- Communication devices: Network card

2. I/O Hardware Structure

Device Controller

- Each I/O device is managed by a **device controller**
- Acts as an interface between the device and the OS
- Contains:
  - Control registers
  - Status registers
  - Data buffers

3. I/O Software Structure

- The OS uses a **layered approach** for I/O handling.

User-Level I/O Software

- Includes system calls like:
  - read()
  - write()
  - open()
  - close()
-

- **Kernel I/O Subsystem**

Provides common services such as:

- Buffering
- Caching
- Spooling
- Error handling

1. Buffering

- **Definition:**
- Buffering uses **temporary memory** to store data during I/O operations.
- **Types:**
- Single buffering
- Double buffering
- Circular buffering
- **Benefits**
- Smooth data transfer
- Improves performance

2. Caching

- Stores frequently accessed data in fast memory
- Reduces access time
- Improves I/O speed

4.Error Handling in I/O

- Detects hardware and software errors
- Uses error codes
- Retries operations if needed

3. Spooling

- **Definition:**
- Spooling (Simultaneous Peripheral Operations On-Line) stores I/O data in a disk buffer.
- **Example:**
- Print spooling
- **Advantages**
- Multiple processes can share a device

Storage Structure

The **storage structure** of an operating system defines **how data and programs are stored, organized, accessed, and managed** in a computer system.
Since different storage devices have different **speed, cost, and capacity**, the OS organizes them in a **hierarchical manner** to achieve high performance and efficient storage utilization.

Types of Storage:

- Primary storage (RAM)
- Secondary storage (Hard disk, SSD)
- Tertiary storage (Backup devices)
- **Storage Hierarchy:**
- Registers → Cache → Main Memory → Secondary Storage
- **Goal:** Balance speed, cost, and capacity

Dual Mode Operation in Operating System

Introduction

- **Dual mode operation** is a protection mechanism used by operating systems to **distinguish between user programs and the operating system code**.
  It ensures that **critical system resources are protected** from unauthorized access by user programs.

Why Dual Mode Operation is Needed

- User programs should **not directly access hardware**
- Prevents **system crashes and security breaches**
- Ensures **safe execution** of programs
- Protects OS kernel from malicious or faulty user code

- **Modes of Operation**
- The CPU operates in **two modes**:
- **1. User Mode**
- Used for executing **user programs**
- Limited access to system resources
- Cannot execute privileged instructions
- Examples of restricted actions:
- Accessing I/O devices
- Modifying memory management registers
- Disabling interrupts

- **2. Kernel Mode (Supervisor / System Mode)**
- Used by the **operating system**
- Full access to hardware and system resources
- Can execute privileged instructions
- Examples of privileged actions:
- I/O operations
- Process scheduling
- Memory management
- Interrupt handling

Mode Bit

- A **mode bit** in the CPU indicates the current mode
- Value:
  - 0 → Kernel mode
  - 1 → User mode
- OS switches the mode bit during execution

Working of Dual Mode Operation

- **Step-by-Step Execution:**
- System starts in **kernel mode**
- OS loads a user program
- CPU switches to **user mode**
- User program executes
- When a system call occurs:
  - Control transfers to OS
  - Mode changes to **kernel mode**
- OS performs requested service
- CPU switches back to **user mode**

- **System Calls and Dual Mode**
- **System calls** provide a safe way for user programs to request OS services
- Examples:
  - read()
  - write()
  - fork()
  - exec()
- User programs **cannot access hardware directly**, they must use system calls.

Dual Mode and Protection

Dual mode operation provides:

- **CPU protection**
- **Memory protection**
- **I/O protection**

It ensures:

- OS integrity
- System stability
- Secure execution environment

System Components

System Components of an Operating System

An Operating System (OS) is made up of several components that work together to manage hardware and software resources and provide services to users and applications.

- Process management
- Main memory management
- File management
- I/O system management
- Secondary storage management
- Protection system
- Command interpreter system
- Networking
- System Call Interface

- **1. Process Management**
- Handles creation, execution, and termination of processes.
- Manages CPU scheduling and process synchronization.
- Prevents deadlocks and ensures efficient CPU usage.
- **Example:** Running multiple applications like a browser, media player, and text editor at the same time

- **2. Main Memory Management**
- Manages primary memory (RAM).
- Keeps track of which memory locations are in use and by which process.
- Allocates and deallocates memory as needed.
- **Example:** Loading a program into RAM for execution.

- **3. File System Management**
- Manages files and directories on storage devices.
- Controls file creation, deletion, reading, and writing.
- Handles access permissions and file protection.
- **Example:** Saving a document or opening a PDF file

- **4. I/O (Input/Output) Device Management**
- Manages communication between devices and the system.
- Uses device drivers to interact with hardware.
- Handles buffering, caching, and spooling.
- **Example:** Printing a document or receiving input from a keyboard.

- **5. Secondary Storage Management**
- Manages storage devices like hard disks and SSDs.
- Handles disk scheduling and free space management.
- Ensures efficient data storage and retrieval.
- **Example:** Installing software on a hard disk.

- **6. Security and Protection**
- Protects system resources from unauthorized access.
- Manages user authentication and access control.
- Ensures data confidentiality and integrity.
- **Example:** Login passwords and file permissions

7. Command Interpreter (Shell)

- Acts as an interface between the user and the OS.
- Interprets user commands and executes them.

Example: Command Prompt in Windows, Bash in Linux.

8. Networking

- Enables communication between computers.
- Manages data transfer over networks.
- Supports resource sharing like files and printers.

Example: Accessing files from a network server.

9. System Call Interface

- Provides a way for programs to request services from the OS.
- Acts as a bridge between user programs and the kernel.

Example: System calls like fork(), read(), write().

- **Diagram (Textual View)**

User

↓

Application Programs

↓

Operating System

- ├── Process Management
- ├── Memory Management
- ├── File System
- ├── I/O Management
- ├── Security

↓

Hardware

Operating System Services

- **Operating System services** are the functions provided by the OS to help **users and programs** execute efficiently and safely.
- 👉 They act as an **interface between the user and the hardware**.

OS provides services to users and programs:

- Program execution
- I/O operations
- File system manipulation
- Communication
- Error detection
- Resource allocation
- Accounting
- Protection and security

- **Major Operating System Services**
- **1. Program Execution**
- Loads a program into memory
- Executes the program
- Handles normal and abnormal termination
- 📌 *Example:* Running a C program or opening a browser.

2. I/O Operations

- Manages input and output devices
- Provides a uniform interface to devices
- Handles buffering and caching
- 📌 *Example:* Reading from keyboard, writing to a file, printing a document.
- **3. File System Manipulation**
- Creates and deletes files
- Reads and writes files
- Manages directories and permissions
- 📌 *Example:* Creating a file using open() or deleting it using rm.

- **4. Communication**
- Enables communication between processes
- Supports shared memory and message passing
- Useful for distributed systems
- 📌 *Example:* Inter-process communication (IPC) using pipes or shared memory.
- **5. Error Detection**
- Detects hardware and software errors
- Takes corrective actions
- Prevents system crash
- 📌 *Example:* Detecting memory access violation.
- **6. Resource Allocation**
- Allocates CPU, memory, I/O devices
- Manages multiple processes efficiently
- Ensures fair resource usage
- 📌 *Example:* CPU scheduling among multiple processes.

- **7. Accounting**
- Keeps track of resource usage
- Monitors CPU time, memory usage
- Helps in billing and performance analysis
- 📌 *Example:* Tracking how much CPU time a process consumes.

- **8. Protection and Security**
- Protects system resources from unauthorized access
- Provides authentication and authorization
- Ensures data security
- 📌 *Example:* User login passwords and file permissions.

- **Operating System Services – Diagram (Textual)**
- User Programs
- |
- Operating System
- |
- ------------------------------
- | Process | Memory | File | I/O |
- ------------------------------
- |
- Hardware

• System Calls

- **System calls** are the **interface between a user program and the operating system**.
  They allow a program to request services from the OS kernel.
- 👉 A user program **cannot directly access hardware**, so it uses system calls

Types of System Calls:

- Process control
- File management
- Device management
- Information maintenance
- Communication
- Protection

Why System Calls are Needed

✔ Access hardware safely
✔ Create and manage processes
✔ Perform file and I/O operations
✔ Enable communication between processes

How System Calls Work (Flow)

- User Program
- |
- System Call
- |
- Operating System (Kernel)
- |
- Hardware

Types of System Calls

1. Process Control System Calls

- Used to create and manage processes.
- **Examples:**
- fork() – create a process
- exec() – execute a new program
- wait() – wait for child process
- exit() – terminate a process
- 📌 *Example:* Creating a child process in UNIX.

2. File Management System Calls

- Used to handle files and directories.
- **Examples:**
- open()
- read()
- write()
- close()
- unlink()
- 📌 *Example:* Writing data into a file.

3. Device Management System Calls

- Used to control I/O devices.
- **Examples:**
- read()
- write()
- ioctl()

📌 *Example:* Reading input from keyboard

4. Information Maintenance System Calls

- Used to get or set system information.
- **Examples:**
- getpid()
- time()
- alarm()
- stat()

📌 *Example:* Getting process ID.

5. Communication System Calls

- Used for inter-process communication (IPC).
- **Examples:**
- pipe()
- shmget()
- mmap()
- send(), recv()
- 📌 *Example:* Data sharing between two processes

6. Protection System Calls

- Used to control access and permissions.
- **Examples:**
- chmod()
- chown()
- umask()
- 📌 *Example:* Changing file permissions.

System Calls vs OS Services

System Calls	OS Services
Low-level interface	High-level functions
Invoked by programs	Provided by OS
Direct kernel interaction	Built using system calls

System Programs

- **System programs** provide a **convenient environment** for program development and execution.
  They are **not part of the kernel**, but they use **system calls** to interact with the operating system.

👉 Think of them as **utilities that make the OS usable**.

Role of System Programs

- Act as an **interface between user and OS**
- Provide tools for **file handling, program execution, communication**
- Make system usage **easy and efficient**

Types of System Programs

- **1. File Management Programs**
- Used to create, delete, copy, rename, and list files.
- **Examples:**
- ls – list files
- cp – copy files
- rm – remove files
- mkdir – create directory
- 📌 *Example:* Managing files in Linux.
-

- **2. Status Information Programs**
- Provide information about the system status.
- **Examples:**
- ps – process status
- top – running processes
- df – disk space usage
- date – system date and time
- 📌 *Example:* Checking memory and CPU usage.

- **3. File Modification Programs**
- Used to modify file contents.
- **Examples:**
- vi, nano – text editors
- sed – stream editor
- 📌 *Example:* Editing a C program.

- **4. Programming Language Support**
- Provide tools for program development.
- **Examples:**
- Compilers: gcc
- Interpreters: python
- Debuggers: gdb
- 📌 *Example:* Compiling a C program.
-

- **5. Program Loading and Execution**
- Used to load and run programs.
- **Examples:**
- Loaders
- Linkers
- Command executors
- 📌 *Example:* Running a compiled executable.

- **6. Communication Programs**
- Enable communication between users and systems.
- **Examples:**
- mail
- ssh
- ftp
- 📌 *Example:* Sending files over a network.

- **7. Background Services (Daemons)**
- Run continuously in the background.
- **Examples:**
- cron
- systemd
- sshd
- 📌 *Example:* Automatic task scheduling

- **System Programs Diagram**
- User
- |
- System Programs
- |
- System Calls
- |
- Operating System Kernel
- |
- Hardware

System Programs vs System Calls

System Programs	System Calls
User-level programs	Kernel-level interface
Use system calls	Provide OS services
Examples: ls, ps	Examples: fork(), read()

• System Structure of an Operating System

- The **system structure of an operating system** defines **how different components of the OS are organized and interact** with each other and with hardware.
- 👉 It shows **how the OS is designed internally**

- **Basic Computer System Structure**
- Users
- |
- Application Programs
- |
- Operating System
- |
- Hardware (CPU, Memory, I/O Devices)

- **Main Components of System Structure**

1. User Level

- Users interact with the system through:
  - Command line
  - GUI
- No direct access to hardware

2. Application Level

- User programs like:
  - Browsers
  - Editors
  - Compilers
- Use **system calls** to access OS services

- **3. Operating System Level**
- Acts as an **intermediary**
- Manages:
  - Processes
  - Memory
  - Files
  - Devices
- Runs in **kernel mode**

- **4. Hardware Level**
- Physical components:
  - CPU
  - RAM
  - Disk
  - I/O devices

Different ways to structure OS:

- Simple Structure
- Layered Structure
- Microkernel
- Modular Structure
- Hybrid Systems

Types of Operating System Structures

1. Simple Structure

- No clear separation of components
- Entire OS in one level
- 📌 *Example:* MS-DOS
- Advantages
- Fast execution
- Disadvantages
- Poor security
- Hard to maintain

- **2. Layered Structure**
- OS divided into layers
- Each layer uses services of lower layer
- User Programs
- Layer 5
- Layer 4
- Layer 3
- Layer 2
- Layer 1
- Hardware
- 📌 *Example:* UNIX (conceptually layered)
- **Advantages**
- Easy debugging
- Modular design
- **Disadvantages**
- Performance overhead
-

- **3. Microkernel Structure**
- Only essential services in kernel
- Others run in user space
- 📌 *Example:* Mach, Minix
- **Advantages**
- High reliability
- Better security
- **Disadvantages**
- Slower due to message passing

- **4. Modular Structure**
- OS has core kernel + loadable modules
- Modules can be added or removed dynamically
- 📌 *Example:* Linux
- **Advantages**
- Flexible
- Efficient

- **5. Hybrid Structure**
- Combination of different structures
- 📌 *Example:* Windows, macOS
- **Advantages**
- Balanced performance and security

Virtual Machines (VMs)

A **Virtual Machine** is a **software-based imitation of a physical computer** that runs its own operating system and applications as if it were a real machine.

👉 One physical system can run **multiple virtual machines**.

Basic Idea of Virtual Machines

- Hardware is shared
- Each VM behaves like an independent system
- Managed by a **Virtual Machine Monitor (VMM)** or **Hypervisor**

Virtual Machine Architecture

- Applications
- Guest Operating System
- **----------**
- Virtual Machine Monitor (Hypervisor)
- **----------**
- Hardware (CPU, Memory, Disk, I/O)

- **Role of Hypervisor (VMM)**
- Creates and manages VMs
- Allocates CPU, memory, storage
- Ensures isolation between VMs
- Handles hardware access

Types of Virtual Machines

1. System Virtual Machines

- Provide a complete system environment
- Each VM runs its own OS
- 📌 *Examples:* VMware, VirtualBox, Hyper-V

2. Process Virtual Machines

- Designed to run a single program
- Exists only during program execution
- 📌 *Examples:* Java Virtual Machine (JVM)

Types of Hypervisors

Type 1 – Bare Metal Hypervisor

- Runs directly on hardware
- High performance
- 📌 *Examples:* VMware ESXi, Xen

Type 2 – Hosted Hypervisor

- Runs on top of an existing OS
- Easy to use
- 📌 *Examples:* VirtualBox, VMware Workstation

Advantages of Virtual Machines

- ✔ Efficient hardware utilization
  ✔ Isolation and security
  ✔ Easy testing and development
  ✔ Run multiple OS on one machine

Disadvantages of Virtual Machines

- ❌ Performance overhead
  ❌ High memory and storage usage
  ❌ Complex management

Real-Time Applications

- Cloud computing
- Server consolidation
- Software testing
- Running legacy applications

Virtual Machines vs Physical Machines

Virtual Machine	Physical Machine
Software-based	Hardware-based
Multiple OS possible	One OS usually
Flexible & scalable	Less flexible

System Design and Implementation

System design and implementation in an operating system deals with **how the OS is planned, structured, and built** to manage hardware and software resources efficiently.

- 👉 It focuses on **what the OS should do (design)** and **how it is built (implementation)**.

System Design

- System design defines the **overall structure and functionality** of the operating system.

Design Goals

- Efficiency
- Reliability
- Security
- Portability
- Scalability
- Maintainability

Design Issues

- **User Goals**
  - Easy to use
  - Fast response
  - Reliable
- **System Goals**
  - Efficient resource utilization
  - Flexibility
  - Easy maintenance

Key Design Decisions

- Choice of **OS structure** (monolithic, layered, microkernel)
- Process management strategy
- Memory management techniques
- File system design
- I/O system design
- Protection and security mechanisms

System Implementation

- System implementation refers to **coding and building the operating system** based on the design

Implementation Languages

- **C language** (most common)
- **Assembly language** (for hardware-specific parts)

📌 Modern OS kernels are mostly written in **C** with small assembly portions

Implementation Issues

- **Performance**
  - Minimize overhead
- **Portability**
  - OS should run on different hardware
- **Security**
  - Prevent unauthorized access
- **Debugging**
- Easier error detection and fixing

Example

- Linux kernel:
  - Designed as a **modular structure**
  - Implemented mainly in **C**
- Windows:
  - Uses **hybrid structure**

Why System Design & Implementation are Important

- ✔ Ensures efficient OS performance
  ✔ Improves reliability and security
  ✔ Makes OS scalable and maintainable
  ✔ Helps adapt OS to new hardware

System Design vs System Implementation

System Design	System Implementation
Planning stage	Coding stage
Defines structure	Builds the OS
Focus on what to do	Focus on how to do

System Generation (SYSGEN)

System generation (SYSGEN) is the process of **configuring and installing an operating system for a specific computer system**.

👉 It customizes the OS according to:

- Hardware configuration
- User requirements

Why System Generation is Needed

- Different systems have different:
- CPU types
- Memory sizes
- I/O devices
- Storage devices
- So, the OS must be **tailored** to work efficiently on each system.

Steps in System Generation

1. Hardware Identification

- Identify CPU type
- Detect memory size
- Identify I/O devices (keyboard, disk, printer)

2. Configuration Selection

- Select required OS components
- Choose device drivers
- Decide scheduling and memory options

3. OS Compilation / Linking

- Compile selected OS modules
- Link them to form the kernel

4. Installation

- Install OS onto disk
- Configure boot loader

5. System Testing

- Test OS functionality
- Verify hardware interaction

Methods of System Generation

1. Manual System Generation

- Administrator configures OS manually
- Time-consuming
- Used in early systems

2. Automatic System Generation

- Uses configuration files or scripts
- Faster and less error-prone
- Used in modern OS
- 📌 *Example:* Linux installation with auto hardware detection.

Advantages of System Generation

- ✔ Efficient use of hardware
  ✔ Optimized OS performance
  ✔ Supports different hardware platforms

Real-Time Example

- Installing Linux on:
  - Laptop
  - Server
  - Virtual machine
- Each installation generates a **customized system**